



COMP9418 – Advanced Topics in Statistical Machine Learning

W3 – Learning in Graphical Models

Instructor: Edwin V. Bonilla

School of Computer Science and Engineering

August 6th, 2018

(Last update: 6/8/18 8:22:48 am)

Acknowledgements

Material derived from:

- [\[Murphy, MLaPP, 2012\]](#) Machine Learning: A Probabilistic Perspective, Kevin P. Murphy, 2012
 - Almost all figures in today's lecture are from this book
- [\[Bishop, PRML, 2006\]](#) Pattern Recognition and Machine Learning, Christopher Bishop, 2006
- [\[Barber, BRML, 2012\]](#) Bayesian Reasoning and Machine Learning, David Barber, 2012
- [\[Bonilla, PML, 2015\]](#) Slides from course to industry, Edwin V. Bonilla, Practical Machine Learning © 2010 – 2017

Aims

This lecture will allow you to understand parameter learning in graphical models. Following it you should be able to:

- Distinguish between inference and learning in graphical models
- Represent full joint distributions using directed graphical models
- Derive and apply maximum likelihood estimators (MLE), maximum a posteriori (MAP) estimators, and carry out full Bayesian inference over parameters in directed graphical models with fully observable data
- Understand the utility of latent variable models in general and mixture models in particular
- Derive and apply expectation-maximisation algorithms to latent variable models

Outline

- I. Intro to Parameter Learning
- II. Learning from Complete Data
- III. Learning with Latent Variables
 - 1. Mixture models
- IV. The Expectation-Maximisation Algorithm
- V. Model Selection

I. Intro to Parameter Estimation

Inference vs Learning

Probabilistic inference: Estimating unknown quantities from known quantities:

- Given X_4 , what is the the distribution over the *latent* variables?

$$P(X_1, X_2, X_3, X_5 | X_4 = x_4) = \frac{P(X_1, X_2, X_3, X_4 = x_4, X_5)}{P(X_4 = x_4)}$$

- Given X_4 , what is the distribution over X_2 ?

$$P(X_2 | X_4 = x_4) = \frac{P(X_2, X_4 = x_4)}{P(X_4 = x_4)}$$

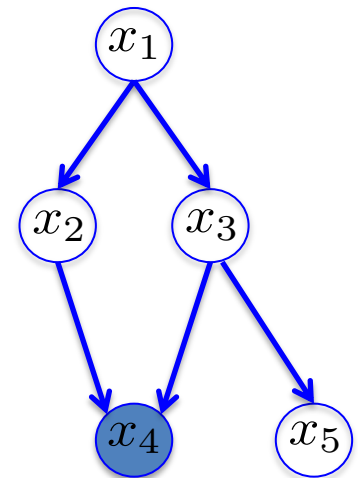
- In general, we are interested in:

query

visible

uninteresting

$$P(\mathbf{X}_q | \mathbf{X}_v) = \sum_{\mathbf{X}_u} P(\underbrace{\mathbf{X}_q, \mathbf{X}_u}_{\text{hidden}} | \mathbf{X}_v) = \frac{\sum_{\mathbf{X}_u} P(\mathbf{X}_q, \mathbf{X}_u, \mathbf{X}_v)}{\sum_{\mathbf{X}_{u'}, \mathbf{X}_{q'}} P(\mathbf{X}_{q'}, \mathbf{X}_{u'}, \mathbf{X}_v)}$$



Inference vs Learning

Parameter Learning (estimation):

- Our distributions are all conditioned on probability tables (parameters) θ , i.e. $P(X_1, \dots, X_D | \theta)$
- Learning is the estimation of these parameters *from data*

- Example: given $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$ $x_j^{(i)} \in \{0, 1\}$

- Estimate $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5$

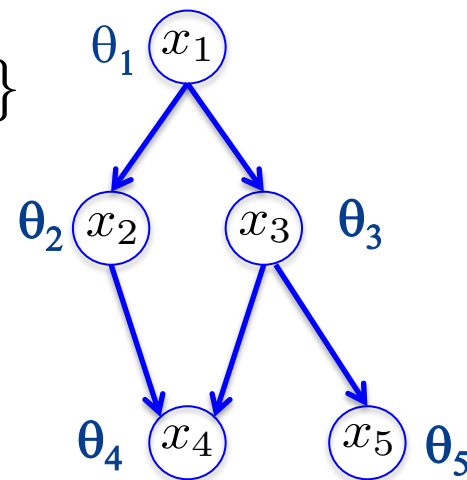
- » How many parameters?

- Can do maximum a posteriori (MAP)

- Or maximum likelihood, assumes uniform prior $P(\theta)$
 - Solve constrained optimisation problem

- From a Bayesian perspective $\rightarrow$ Inference problem

- Estimate posterior distribution over θ given the data



The Full Graphical Model

- Data points conditionally independent given θ
 - But marginally dependent
 - $P(\mathbf{x}^{(i)} | \theta)$ is the same for all $i = 1, \dots, N$ (given by a Bayesian network)
 - *Exchangeability*: Order of the data does not matter

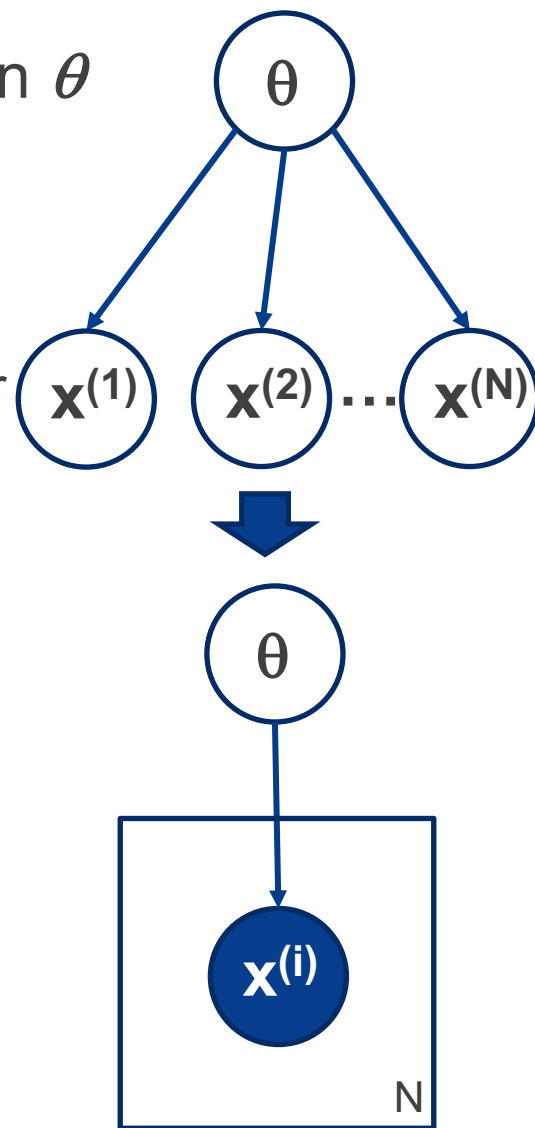


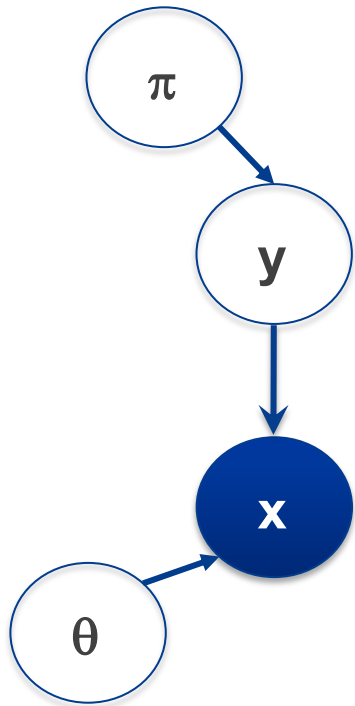
Plate notation

- Nodes within the box will get repeated when the model is *unrolled*

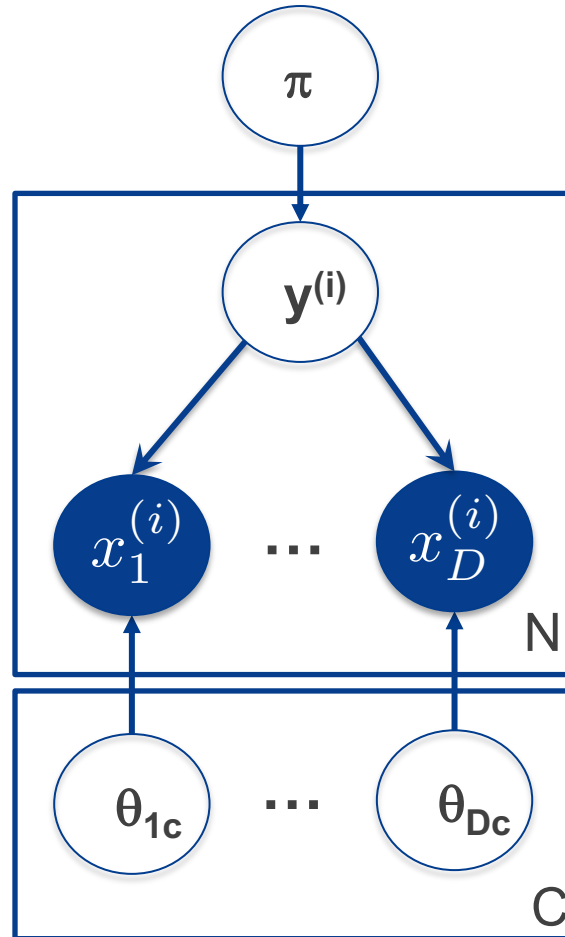
Joint distribution

$$p(\boldsymbol{\theta}, \mathcal{D}) = p(\boldsymbol{\theta}) \left[\prod_{i=1}^N p(\mathbf{x}^{(i)} | \boldsymbol{\theta}) \right]$$

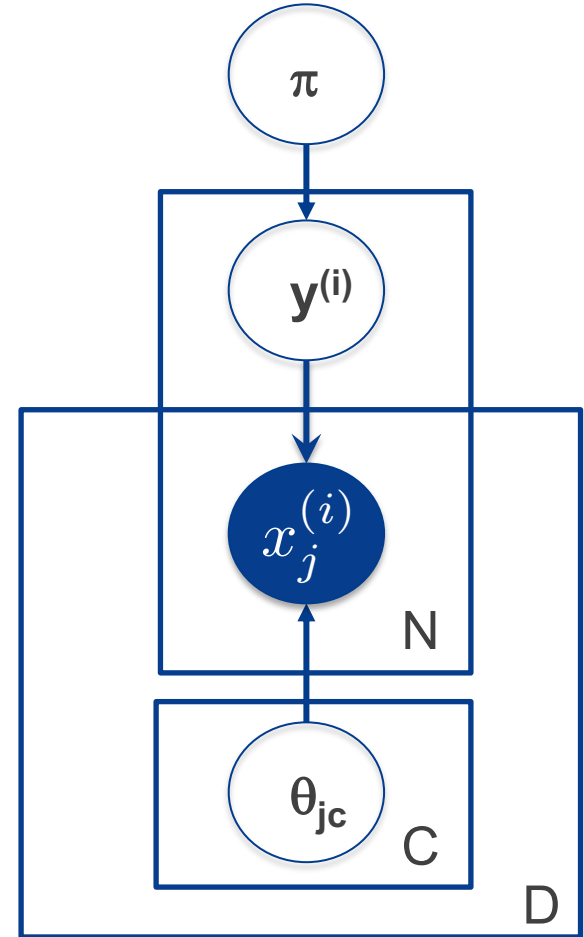
Full Naïve Bayes Model



Generative model. Not showing all data points or structure in features.



Showing all variables with plates but features unrolled.



Compact representation using nested plates.

II. Learning From Complete Data

Some Notational Convenience

Recall we defined the categorical distribution for one-hot-encoding of $\mathbf{x}$ as

$$\text{Cat}(\mathbf{x}|\boldsymbol{\theta}) = \prod_{k=1}^K \theta_k^{x_k} \quad x_k \in \{0, 1\}, \quad \sum_{k=1}^K x_k = 1$$

A slightly more convenient notation when $x \in \{1, \dots, K\}$ is:

$$\text{Cat}(x|\boldsymbol{\theta}) = \prod_{k=1}^K \theta_k^{\mathbb{I}(x=k)}$$

Where θ_k is the probability of category k .

- We can use either as long as we explicitly say what encoding we are using for the categorical variable:
 - One-hot-encoding
 - The actual value of the category

Learning in Directed Graphical Models

Given $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}$ and Bayesian network structure, we want to estimate the parameters θ

- **Complete data:** All variables fully observed

- No missing data
- No latent variables in the models

- **Likelihood:**

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{i=1}^N p(\mathbf{x}^{(i)}|\boldsymbol{\theta}) \quad \text{iid assumption} = \prod_{i=1}^N \prod_{t=1}^V p(x_t^{(i)} | x_{\pi(t)}^{(i)}, \boldsymbol{\theta}_t) \quad \begin{array}{l} \text{Product over} \\ \text{nodes} \end{array} = \prod_{t=1}^V p(\mathcal{D}_t | \boldsymbol{\theta}_t) \quad \begin{array}{l} \text{Parents of} \\ \text{node } t \end{array} \quad \begin{array}{l} \text{CPTs of} \\ \text{node } t \end{array} \quad \begin{array}{l} \text{Data associated with} \\ \text{note } t \text{ and its parents} \end{array}$$

- **Prior:** Assume factorization over CPTs $p(\boldsymbol{\theta}) = \prod_{t=1}^V p(\boldsymbol{\theta}_t)$

What likelihood model can we assume for discrete (categorical) data?

Full Specification of the Model

Let x_t be a categorical variable $x_t \in \{1, \dots, K_t\}$ then we can assume that:

$$p(x_t | \mathbf{x}_{\pi(t)} = c) = \text{Cat}(x_t | \boldsymbol{\theta}_{tc})$$

Where: $\theta_{tck} \stackrel{\text{def}}{=} p(x_t = k | \mathbf{x}_{\pi(t)} = c)$

Observed
category for
node t

Observed parent
configuration for node t

Number of parent
combinations

$$t = 1, \dots, V \quad c = 1, \dots, C_t$$

$$k = 1, \dots, K_t$$

$$C_t \stackrel{\text{def}}{=} \prod_{s \in \pi(t)} K_s$$

Obviously, by normalisation:

$$\sum_{k=1}^{K_t} \theta_{tck} = 1$$

Parameter learning

- Maximum a posteriori (MAP)

- Maximise $\mathcal{L}_{\text{MAP}}(\boldsymbol{\theta}) = \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta})$

s.t: $0 \leq \theta_{tck} \leq 1$ and $\sum_{k=1}^{K_t} \theta_{tck} = 1$

- If $p(\boldsymbol{\theta})$ is uniform $\rightarrow$ maximum likelihood solution

$$\hat{\theta}_{tck} = \frac{N_{tck}}{\sum_{k'=1}^{K_t} N_{tck'}} \quad \text{How did we get this?}$$

where
$$N_{tck} = \sum_{i=1}^N \mathbb{I}(x_t^{(i)} = k, x_{\pi(t)}^{(i)} = c)$$
 Number of times feature (node) t took value k and its parents took configuration value c

Can be fully Bayesian?

Bayesian “Estimation” of Parameters

- Both the likelihood and the prior factorise:

$$p(\mathcal{D}|\boldsymbol{\theta}) = \prod_{t=1}^V p(\mathcal{D}_t|\boldsymbol{\theta}_t) \quad p(\boldsymbol{\theta}) = \prod_{t=1}^V p(\boldsymbol{\theta}_t)$$

- Hence the **posterior** also factorises:

$$p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})$$

What prior can we use?

- We can place a separate Dirichlet prior over each CPT:

$$p(\boldsymbol{\theta}_{tc}) = \text{Dir}(\boldsymbol{\theta}_{tc}|\boldsymbol{\alpha}_{tc})$$

- Naturally yields a Dirichlet posterior

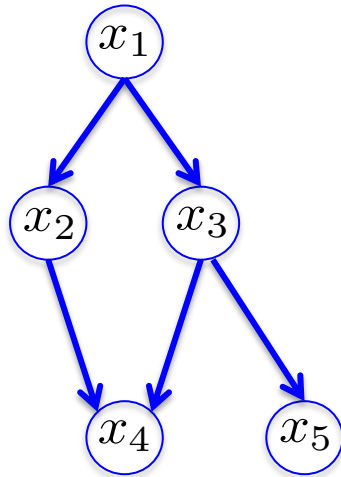
$$p(\boldsymbol{\theta}_{tc}|\mathcal{D}, \boldsymbol{\alpha}_{tc}) = \text{Dir}(\boldsymbol{\theta}_{tc}|\mathbf{N}_{tc} + \boldsymbol{\alpha}_{tc})$$

- Where N_{tck} is defined as before.

↑
Smoothed estimate
with pseudo-counts

Example of Parameter Learning in DGMs

Given the following directed graphical model and binary data:



		Variables				
		x_1	x_2	x_3	x_4	x_5
Data points	$\mathbf{x}^{(1)}$	0	0	1	0	0
	$\mathbf{x}^{(2)}$	0	1	1	1	1
	$\mathbf{x}^{(3)}$	1	1	0	1	0
	$\mathbf{x}^{(4)}$	0	1	1	0	0
	$\mathbf{x}^{(5)}$	0	1	1	1	0

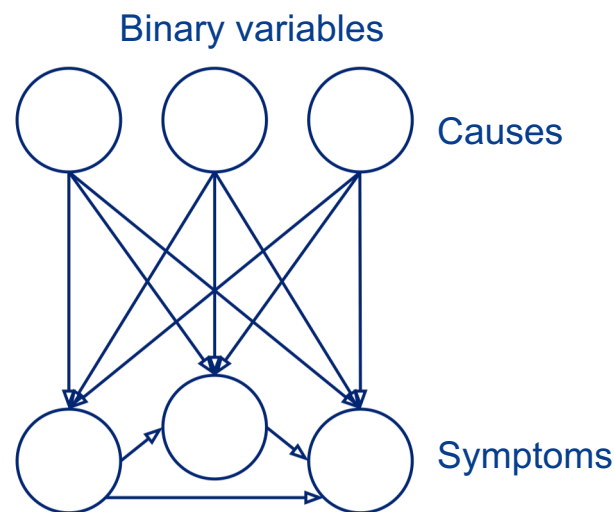
Assuming $\alpha_{tck} = 1$, the posterior mean for $t=4$ is: (Mean of $p(\theta_{4ck}|\mathcal{D})$)

Parent configurations	x_2	x_3	$N_{tck=1}$	$N_{tck=0}$	$\bar{\theta}_{tck=1}$	$\bar{\theta}_{tck=0}$
	0	0	0	0	1/2	1/2
	1	0	1	0	2/3	1/3
	0	1	0	1	1/3	2/3
	1	1	2	1	3/5	2/5

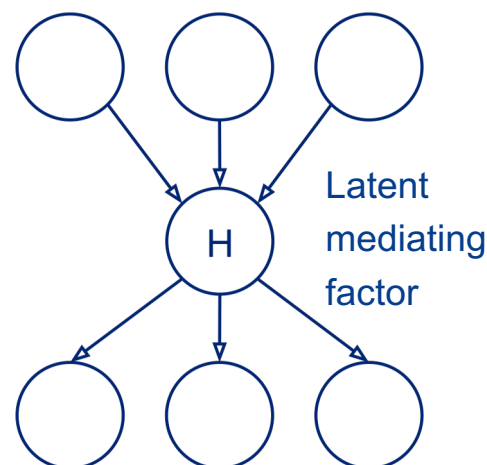
III. Learning with Latent Variables and Mixture Models

Latent Variable Models

- Graphical models define joint probability distributions
 - Direct dependences defined by connecting nodes with edges
- Alternatively, we can define dependences via hidden “causes”
 - Models with hidden variables are known as **latent variable models (LVMs)**
 - » These models are harder to learn
 - » **Why are LVMs useful?**



How many parameters? 59

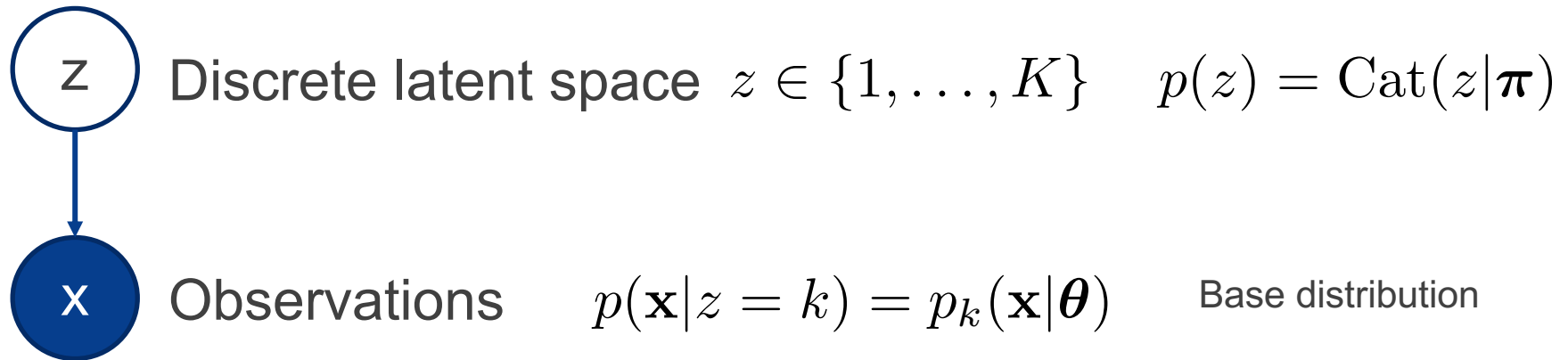


How many parameters? 17

Latent Variables:

- Model dependences efficiently
- Compressed representation of the data

Mixture Models



The distribution of the observations is a **convex combination** of the base distribution (marginal of **x**):

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k p_k(\mathbf{x}|\boldsymbol{\theta}) \quad \text{with} \quad 0 \leq \pi_k \leq 1 \quad \text{and} \quad \sum_{k=1}^K \pi_k = 1$$

Two blue arrows point from the words "convex combination" in the text above to the coefficients π_k in the equation.

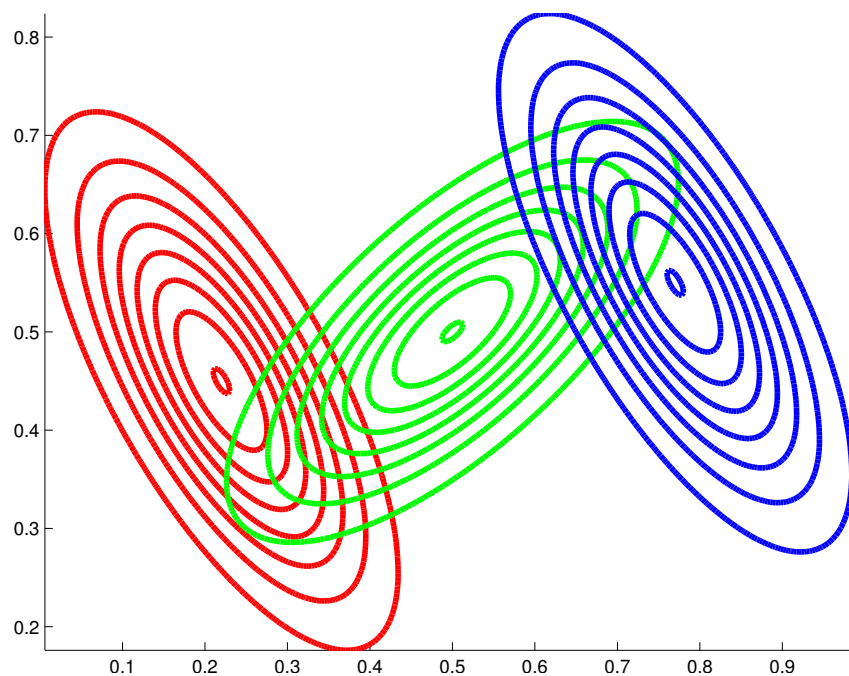
Base distribution can be of any type

Example: Mixture of Gaussians (MoG)

$\mathbf{x} \in \mathbb{R}^D$: Each base distribution is a multivariate Gaussian

mean covariance

$$p_k(\mathbf{x}|\boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad \text{Aka Gaussian Mixture Model (GMM)}$$



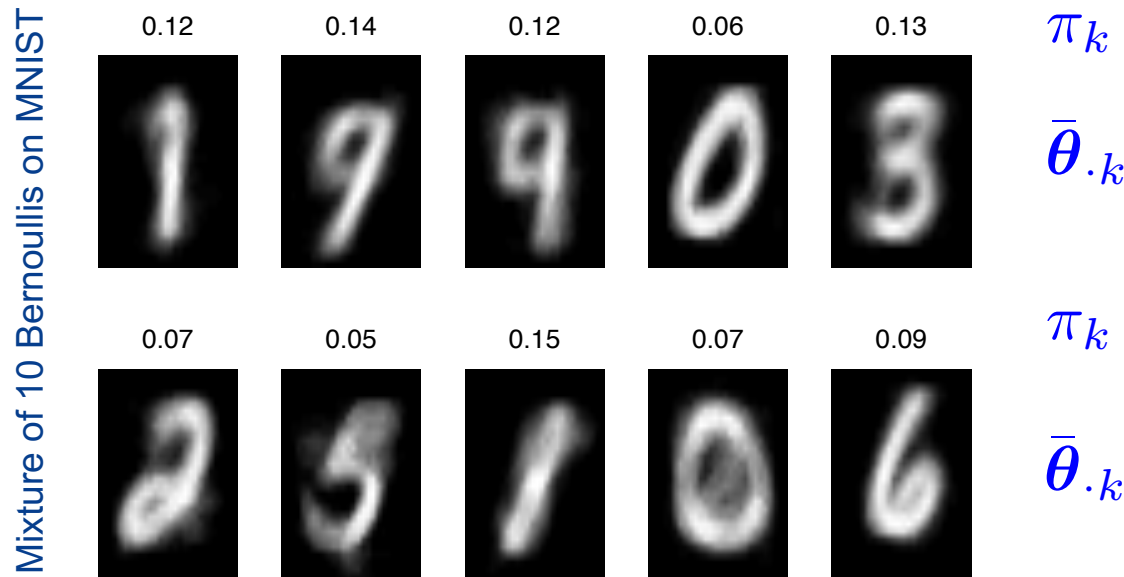
Given a sufficiently large number of mixture components, a GMM can be used to approximate any density defined on $\mathbb{R}^D$

Example: Mixture of Product of Bernoullis

$x_j \in \{0, 1\}$: Each base distribution is a product of Bernoulli distributions:

$$p_k(\mathbf{x}|\boldsymbol{\theta}) \stackrel{\text{def}}{=} p(\mathbf{x}|z = k) = \prod_{j=1}^D \text{Ber}(x_j|\theta_{jk})$$

- Given the latent assignment, the features are conditionally independent
- However, the observed features are (marginally) dependent!



Applications of Mixture Models

- Flexible density estimation

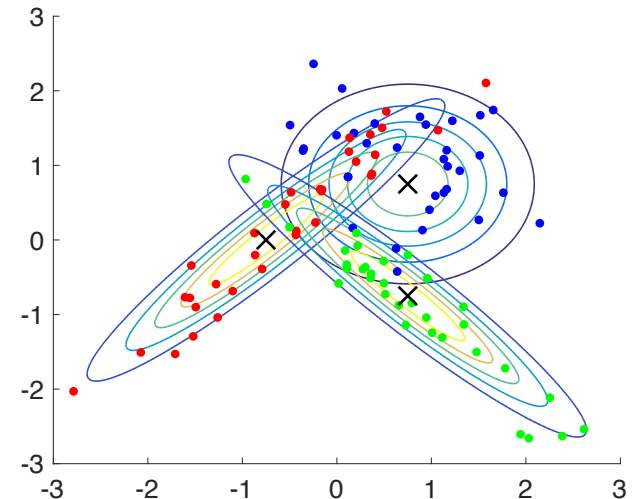
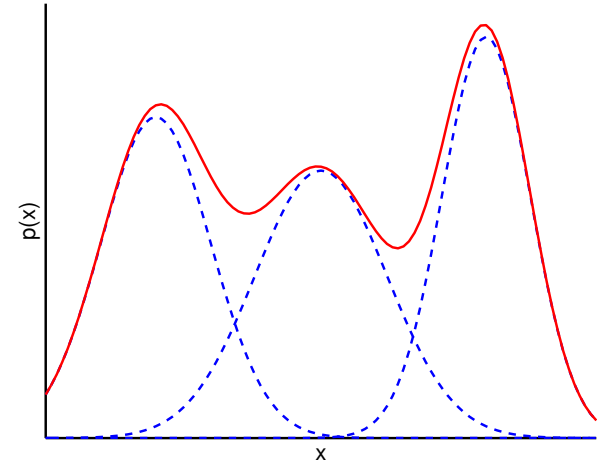
- Multimodal distributions
- Data compression
- Outlier detection
- Generative classifiers

- Clustering

1. Fit mixture model
2. Compute posterior

$$p(z^{(i)} = k | \mathbf{x}^{(i)}, \boldsymbol{\theta}, \boldsymbol{\pi}) \propto p(z^{(i)} = k | \boldsymbol{\pi}) p(\mathbf{x}^{(i)} | z^{(i)} = k, \boldsymbol{\theta})$$

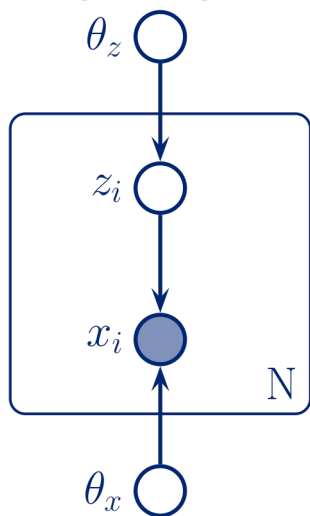
- » Probability of $\mathbf{x}^{(i)}$ belonging to cluster k
- » aka responsibility
- » Soft clustering
- » Label switching (can permute labels)



Parameter Estimation in LVMs is Hard

- When having complete data:
 - Factorised likelihood and factorised prior $\rightarrow$ Factorised posterior
 - Global optimum for MAP or maximum likelihood estimator (MLE)

- When having latent variables:



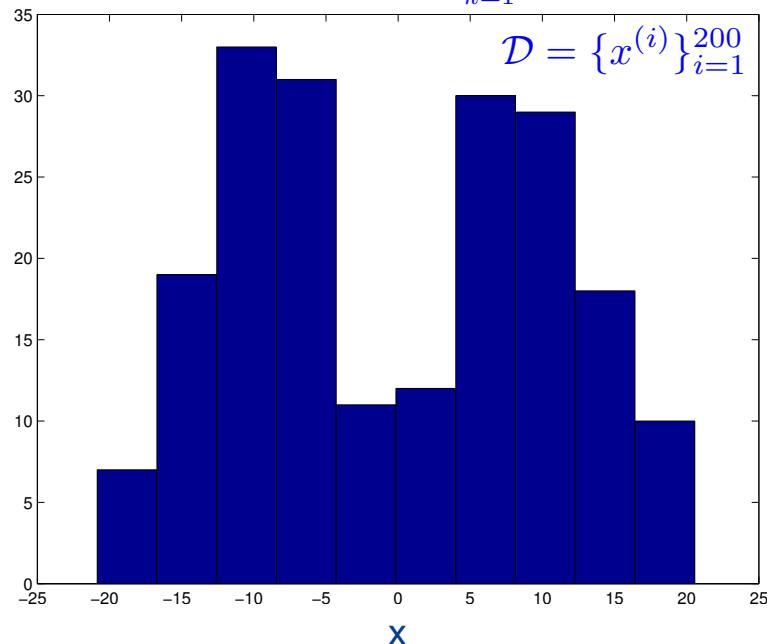
- If z_i were observed $\rightarrow \theta_z \perp\!\!\!\perp \theta_x | \mathcal{D}$
- However, z_i are latent, then $\theta_z \not\perp\!\!\!\perp \theta_x | \mathcal{D}$
 - Posterior (over parameters) does not factorise
- Multimodal posterior over parameters
 - Different labelling of the clusters
 - Even MAP and MLE are hard (non-convex)

- The parameters are **not identifiable**
 - Not a unique MLE

An Example of Unidentifiability

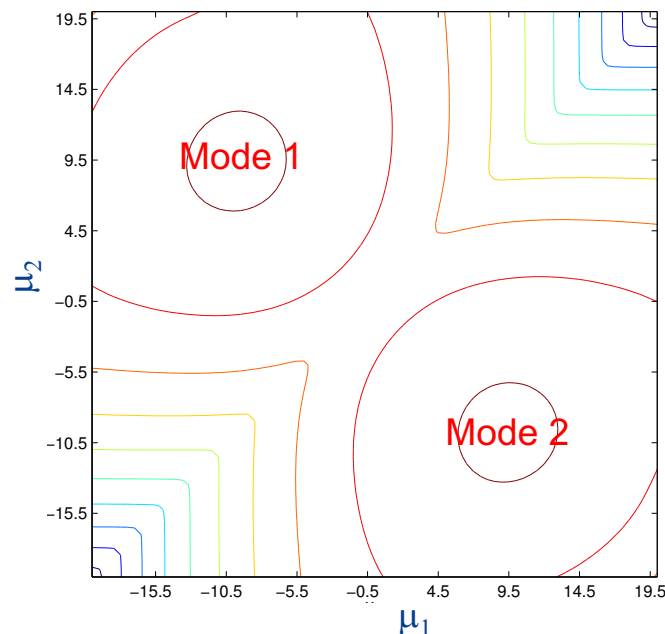
A 1d GMM with $\pi_k = 0.5$, $\sigma_k = 5$,
 $\mu_1 = -10$, $\mu_2 = 10$

samples from $p(x|\boldsymbol{\pi}, \boldsymbol{\mu}) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \sigma_k^2)$



Fix $\pi_k = 0.5$, $\sigma_k = 5$

Likelihood surface $p(\mathcal{D}|\mu_1, \mu_2)$



- Why is identifiability an issue for parameter estimation?
 - Interpretability (e.g. clusters), sampling, non-convex optimization
 - Sometimes we will be happy with a local optimum (approximate MAP)

IV. The Expectation Maximisation (EM) Algorithm

General Idea of Expectation-Maximisation (EM)

- Recall that for complete data: $\mathcal{L}_{\text{MAP}}(\boldsymbol{\theta}) = \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\boldsymbol{\theta})$
 - Parameter estimation via gradient-based optimization
 - MAP easy to compute
- Models with latent variables (incomplete data) are harder
 - If there were complete data $\rightarrow$ MAP easy
- Iterate:
 1. [E-step] Infer missing values given current parameters
 2. [M-step] Optimize parameters given the “filled-in” data
- Nice properties
 - Often closed-form updates for both steps
 - Constraints on parameters automatically satisfied
 - Monotonically increases the log likelihood of the (observed data) at each iteration

General EM Algorithm (1)

$\mathbf{x}^{(i)}$: Observed variable for data point $i \rightarrow \mathbf{X} = \{\mathbf{x}^{(i)}\}$

$\mathbf{z}^{(i)}$: Hidden or missing variable $\rightarrow \mathbf{Z} = \{\mathbf{z}^{(i)}\}$

Goal: maximize the data log likelihood:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^N \log p(\mathbf{x}^{(i)} | \boldsymbol{\theta}) = \sum_{i=1}^N \log \sum_{\mathbf{z}^{(i)}} p(\mathbf{x}^{(i)}, \mathbf{z}^{(i)} | \boldsymbol{\theta})$$

- Can be hard to optimise using gradient-based approaches
 - » Note logarithm of the sum
 - » Enforce constraints on parameters

Instead, define the **complete data log likelihood**:

$$\mathcal{L}_c(\boldsymbol{\theta}) \stackrel{\text{def}}{=} \sum_{i=1}^N \log p(\mathbf{x}^{(i)}, \mathbf{z}^{(i)} | \boldsymbol{\theta}) \quad \text{This cannot be computed since } \mathbf{z}^{(i)} \text{ are hidden}$$

What can we do? Take expectations over hidden variables

General EM Algorithm (2)

- Define the **auxiliary function**: $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) = \mathbb{E}_{p(\mathbf{Z}|\mathbf{X}, \boldsymbol{\theta}^{\text{old}})} [\mathcal{L}_c(\boldsymbol{\theta})]$
 - $\boldsymbol{\theta}^{\text{old}}$: Parameters at previous iteration
 - Expectation over the posterior
 - Complete data log likelihood

1. E-step: Compute $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}})$

- Compute posterior $p(\mathbf{Z} | \mathbf{X}, \boldsymbol{\theta}^{\text{old}})$ as a function of (old) parameters
- Expected sufficient statistics

2. M-step: Optimize $Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}})$ wrt $\boldsymbol{\theta}$:

$$\boldsymbol{\theta}^{\text{new}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) : \text{Parameters at new iteration}$$

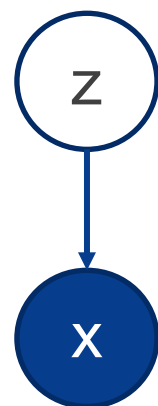
- For MAP estimation the E-step remains unchanged and we modify the M-step:

$$\boldsymbol{\theta}^{\text{new}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \left[Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) + \log p(\boldsymbol{\theta}) \right]$$

- Likelihood of observed data guaranteed not to decrease*
 - *Useful debugging tool*

Gaussian Mixture Models (GMMs)

Recall a GMM with K components is defined as:



Discrete latent space $z \in \{1, \dots, K\}$

$$p(z) = \text{Cat}(z|\boldsymbol{\pi}) \quad 0 \leq \pi_k \leq 1$$

$$\sum_{k=1}^K \pi_k = 1$$

Observations

$$p(\mathbf{x}|z = k) = p_k(\mathbf{x}|\boldsymbol{\theta}) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

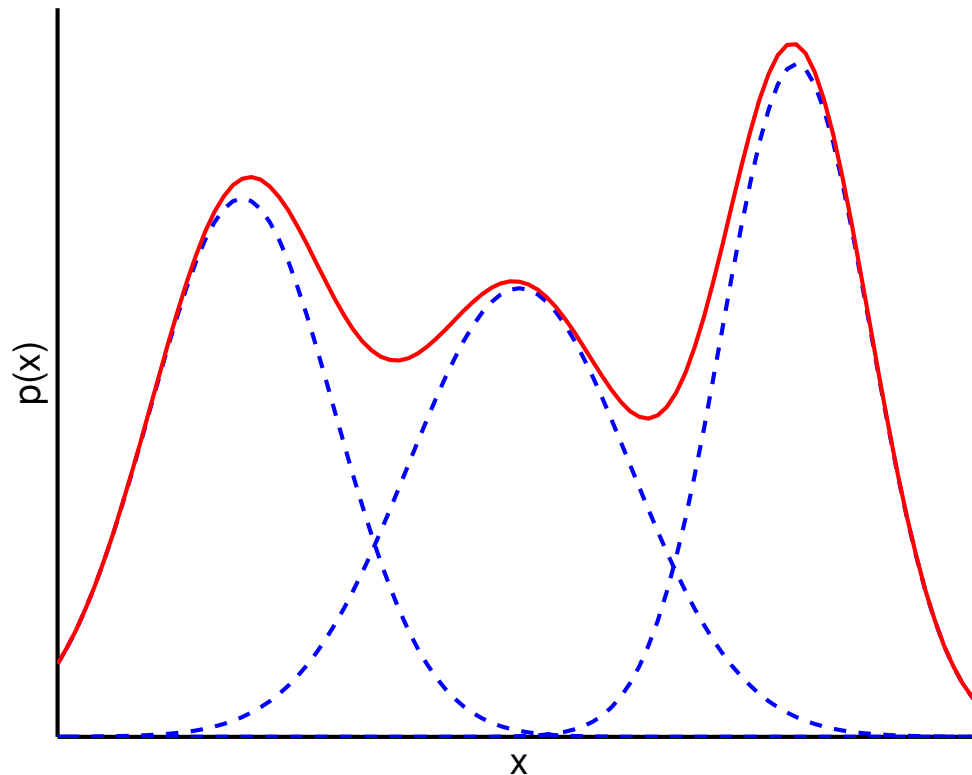
Hence the marginal:
$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

Goal: Estimate model parameters $\boldsymbol{\theta} = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$ from data

– Constraints on π_k and $\boldsymbol{\Sigma}_k$?

Gaussian Mixture Models (GMMs)

1D example



- In **blue**: original Gaussians scaled by their priors $p(z_k = 1)$ with $k = 1, 2, 3$
- In **red**: the resulting mixture
- The final density is multimodal

Generative Process: For $k = 1, 2, 3$

1. Pick component (cluster) k with probability π_k
2. Draw $\mathbf{x} \sim N(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$

EM for GMMs (1)

E-step: Auxiliary function

$$\begin{aligned} Q(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) &= \mathbb{E}_{p(\mathbf{z}|\mathbf{X}, \boldsymbol{\theta}^{\text{old}})} [\mathcal{L}_c(\boldsymbol{\theta})] \\ &= \mathbb{E}_{p(\mathbf{z}|\mathbf{X}, \boldsymbol{\theta}^{\text{old}})} \left[\sum_{i=1}^N \log p(\mathbf{x}^{(i)}, z^{(i)} | \boldsymbol{\theta}) \right] \\ &= \sum_{i=1}^N \mathbb{E}_{p(z^{(i)} | \mathbf{x}^{(i)}, \boldsymbol{\theta}^{\text{old}})} \left[\log \left(\prod_{k=1}^K \left(\pi_k p_k(\mathbf{x}^{(i)} | \boldsymbol{\theta}_k) \right)^{\mathbb{I}(z^{(i)}=k)} \right) \right] \\ &= \sum_{i=1}^N \sum_{k=1}^K \mathbb{E}_{p(z^{(i)} | \mathbf{x}^{(i)}, \boldsymbol{\theta}^{\text{old}})} \left[\mathbb{I}(z^{(i)} = k) \right] \log \left(\pi_k p_k(\mathbf{x}^{(i)} | \boldsymbol{\theta}_k) \right) \\ &= \sum_{i=1}^N \sum_{k=1}^K \underbrace{p(z^{(i)} = k | \mathbf{x}^{(i)}, \boldsymbol{\theta}^{\text{old}})}_{r_k^{(i)}} \log \left(\pi_k p_k(\mathbf{x}^{(i)} | \boldsymbol{\theta}_k) \right) \end{aligned}$$

$r_k^{(i)}$ Responsibility of cluster k for data point i

EM for GMMs (2)

E-step: Responsibilities – Posterior cluster assignments:

$$\begin{aligned} r_k^{(i)} &\stackrel{\text{def}}{=} p(z^{(i)} = k | \mathbf{x}^{(i)}, \boldsymbol{\theta}^{\text{old}}) \\ &= \frac{p(z^{(i)} = k | \boldsymbol{\theta}^{\text{old}}) p(\mathbf{x}^{(i)} | z^{(i)} = k, \boldsymbol{\theta}^{\text{old}})}{p(\mathbf{x}^{(i)} | \boldsymbol{\theta}^{\text{old}})} \\ &= \frac{\pi_k^{\text{old}} \mathcal{N}(\mathbf{x}^{(i)} | \boldsymbol{\mu}_k^{\text{old}}, \boldsymbol{\Sigma}_k^{\text{old}})}{\sum_{j=1}^K \pi_j^{\text{old}} \mathcal{N}(\mathbf{x}^{(i)} | \boldsymbol{\mu}_j^{\text{old}}, \boldsymbol{\Sigma}_j^{\text{old}})} \end{aligned}$$

- π_k is the prior probability of component k
- $r_k^{(i)}$ is the posterior probability of component k given that we have observed $\mathbf{x}^{(i)}$
 - can be viewed as the responsibility that component k takes for explaining observation $\mathbf{x}$

EM for GMMs (3)

M-step: Maximise the auxiliary function

$$\boldsymbol{\theta}^{\text{new}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}})$$

Where:
$$\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}}) = \sum_{i=1}^N \sum_{k=1}^K r_k^{(i)} \log \left(\pi_k \mathcal{N}(\mathbf{x}^{(i)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

- $\boldsymbol{\theta} = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$
- Note responsibilities are fixed

Optimizing $\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}})$ wrt $\boldsymbol{\theta}$ we obtain: *How?*

$$\hat{\pi}_k = \frac{r_k}{N} \quad \hat{\boldsymbol{\mu}}_k = \frac{1}{r_k} \sum_{i=1}^N r_k^{(i)} \mathbf{x}^{(i)} \quad \hat{\boldsymbol{\Sigma}}_k = \frac{1}{r_k} \sum_{i=1}^N r_k^{(i)} (\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}^{(i)} - \hat{\boldsymbol{\mu}}_k)^T$$

- where $r_k \stackrel{\text{def}}{=} \sum_{i=1}^N r_k^{(i)}$ “weighted number of points” assigned to cluster k

- *Interpretation of these estimates?*

We then make $\boldsymbol{\theta}^{\text{old}} = \{\hat{\pi}_k, \hat{\boldsymbol{\mu}}_k, \hat{\boldsymbol{\Sigma}}_k\}$ and go to the next E-step

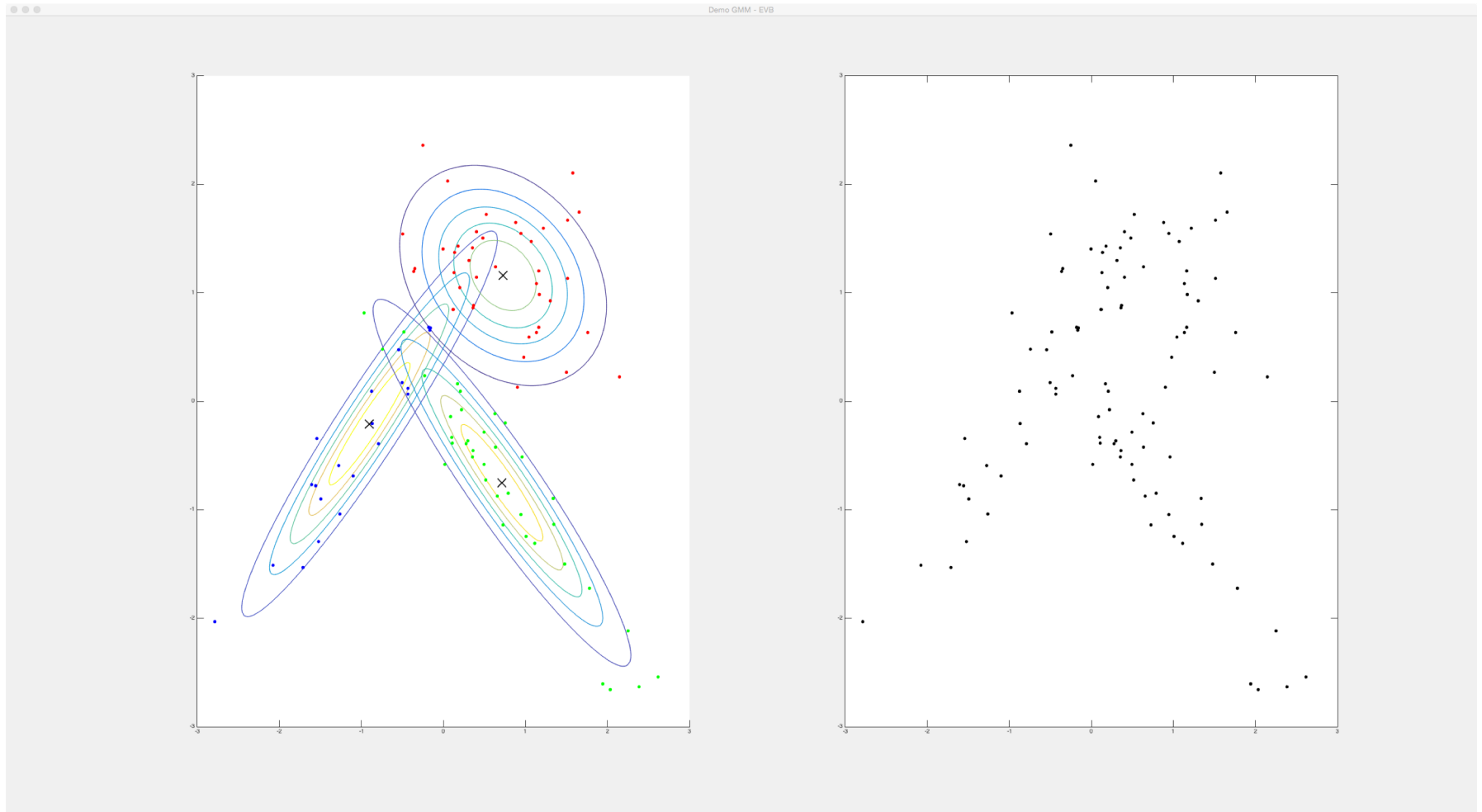
EM for GMMs (4) – The Algorithm

Goal: Given $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ and K learn a GMM

Input: $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N, K$

1. Initialise $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$ (how?) and evaluate data log-likelihood
2. Repeat until convergence of parameters or data log-likelihood
 - a. **E-step:** Compute responsibilities $r_k^{(i)}$ using current parameters $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$
 - b. **M-step:** Re-estimate $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$ using current (fixed) responsibilities $r_k^{(i)}$
 - c. Compute data log-likelihood with current parameters and evaluate convergence

Learning GMMs: Demo



demo_gmm.m

Why EM Works (1)

- EM is a general learning framework useful when:
 - We are dealing with latent variables
 - Direct likelihood maximization is hard

- Consider the Kullback-Leibler (KL) divergence:

$$\text{KL}(q(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta})) \stackrel{\text{def}}{=} \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \left[\log \frac{q(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta})} \right] \geq 0$$

Approximating posterior True (but unknown) posterior Unknown parameters

- With equality occurring iif $q(\mathbf{z}|\mathbf{x}) = p(\mathbf{z}|\mathbf{x}, \boldsymbol{\theta})$
 - We can show that EM optimizes a lower bound $\mathcal{L}_{\text{lower}}(q, \boldsymbol{\theta})$ of the true marginal likelihood, i.e. $\log p(\mathbf{X}|\boldsymbol{\theta}) \geq \mathcal{L}_{\text{lower}}(q, \boldsymbol{\theta})$
 - To optimise the lower bound for fixed parameters we make:

$$q(\mathbf{Z}|\mathbf{X}) = p(\mathbf{Z}|\mathbf{X}, \boldsymbol{\theta}^{\text{old}})$$

E-step why?

- To optimise the lower bound for fixed $q(\mathbf{Z}|\mathbf{X})$

- Maximise. $\mathcal{Q}(\boldsymbol{\theta}, \boldsymbol{\theta}^{\text{old}})$ wrt $\boldsymbol{\theta}$ M-step

Why EM Works (2) – Illustration

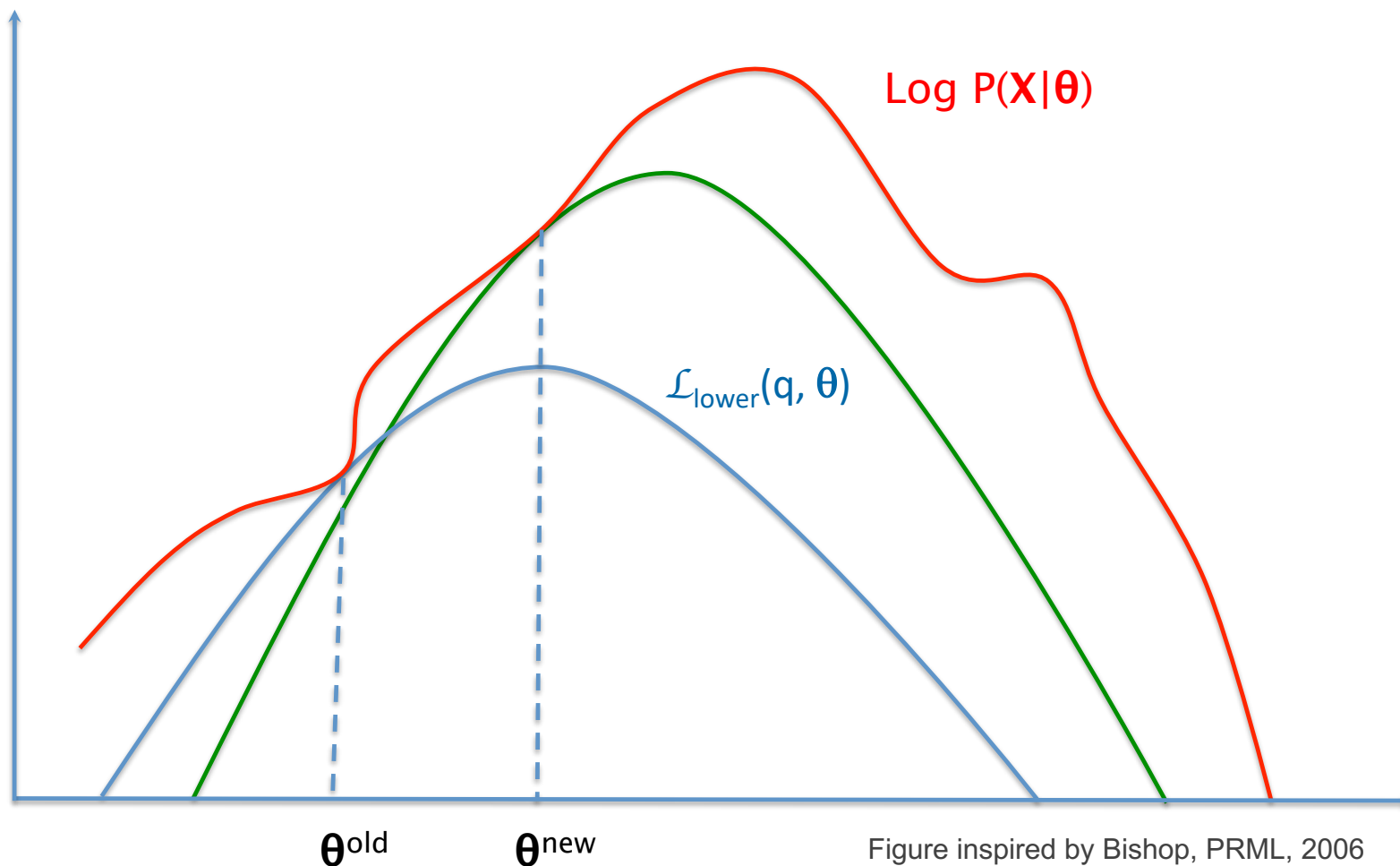


Figure inspired by Bishop, PRML, 2006

Iteratively, compute lower bound for fixed parameters θ^{old} and then maximize this lower bound obtaining θ^{new}

Relation Between GMMs and K-means

- GMM does soft clustering - we have posterior $p(z_k^n = 1 | \mathbf{x})$
 - K-means does hard clustering
- Consider a GMM model with $\Sigma_k = \sigma^2 \mathbf{I}$
 - We obtain a **softmax** function for the responsibilities:

$$r_k^{(i)} = \frac{\pi_k \exp \left(-\frac{1}{2\sigma^2} \|x^{(i)} - \boldsymbol{\mu}_k\|^2 \right)}{\sum_{j=1}^K \pi_j \exp \left(-\frac{1}{2\sigma^2} \|x^{(i)} - \boldsymbol{\mu}_j\|^2 \right)}$$

- The (hard) max is obtained by making $\sigma^2 \rightarrow 0$
- Then $r_k^{(i)} \rightarrow \{0, 1\}$, i.e. hard assignment
 - Cluster centers given by $\boldsymbol{\mu}_k$
- Optimizing complete data log-likelihood same as optimizing error function in K-means
 - **K-means is the hard limit of GMMs**

Singularities in GMMs and Other Issues

- **Singularities** – Collapsing variance problem

- Consider the simple case $\Sigma_k = \sigma_k^2 \mathbf{I}$, $K=2$
- Centre one of the components to a single data point $\mu_2 = \mathbf{x}^{(1)}$

$$\mathcal{N}(\mathbf{x}^{(1)} | \mu_2, \sigma_2^2 \mathbf{I}) = \frac{1}{\sqrt{2\pi\sigma_2^2}} \exp^0$$

- Make $\sigma_2 \rightarrow 0 \rightarrow$ **the log-likelihood will tend to infinity!**

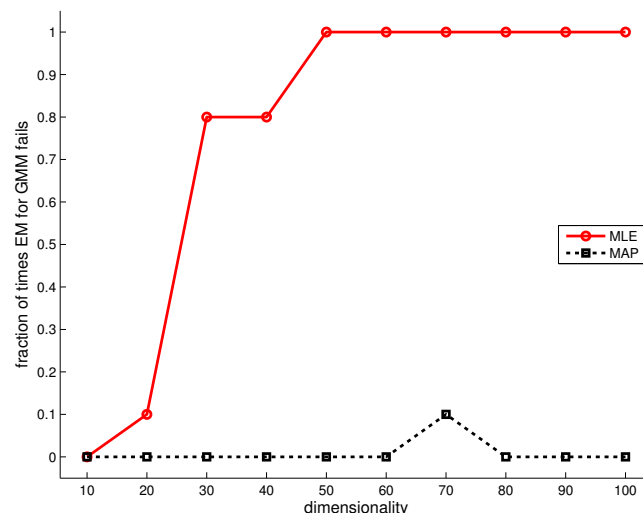
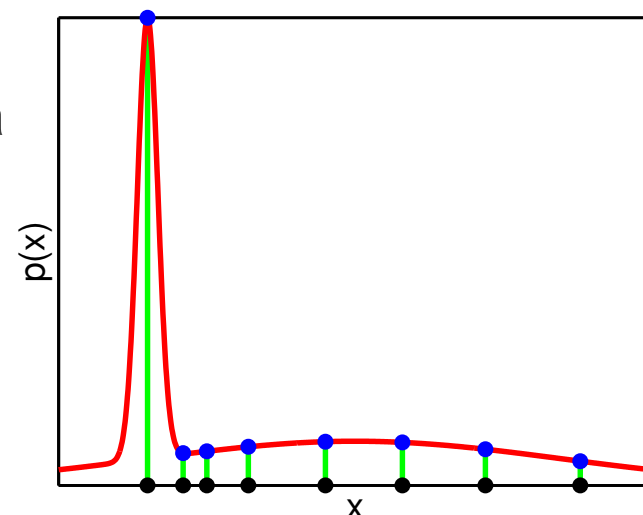
- Solution: do MAP estimation

$$\theta^{\text{new}} = \underset{\theta}{\operatorname{argmax}} \left[\mathcal{Q}(\theta, \theta^{\text{old}}) + \log p(\theta) \right]$$

- **Identifiability**

- **Local optima**

- susceptible to different initializations
 - » We can use K-means



V. Model Selection

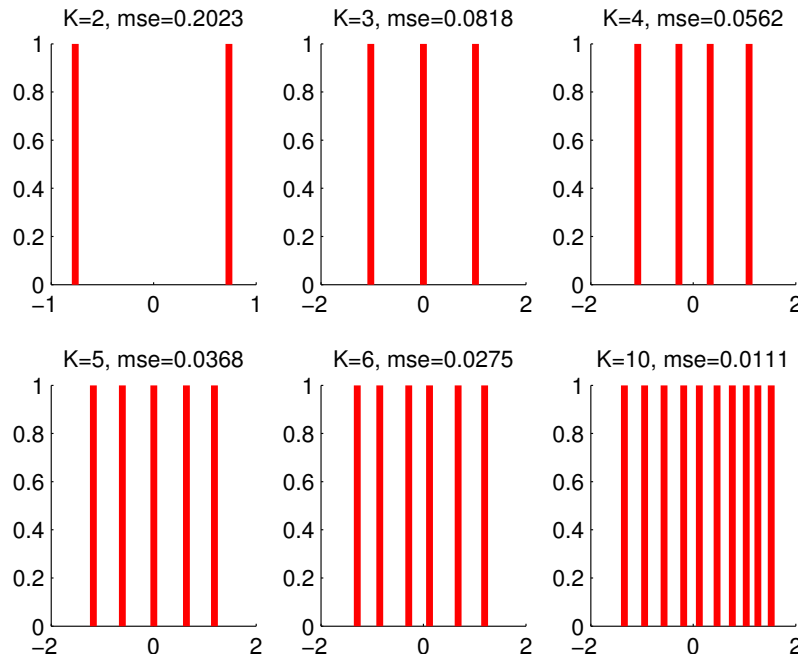
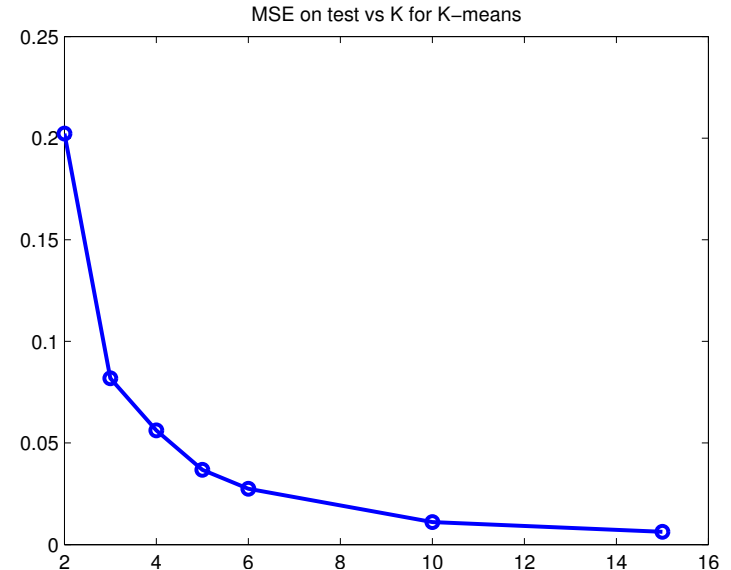
Model Selection for Latent Variable Models

Probabilistic models

- How many latent variables? E.g. number of clusters K
 - Pick model with largest *marginal* likelihood $K^* = \operatorname{argmax} p(\mathcal{D}|K)$
 - Evaluating marginal likelihood can be hard
- Use cross-validation
 - Likelihood on validation set as performance measure
 - Can be computationally demanding
- Other approaches
 - Sampling in the space of models (e.g. using MCMC)
 - Non-parametric priors, e.g. Dirichlet processes
 - » Allow for an unbounded number of components

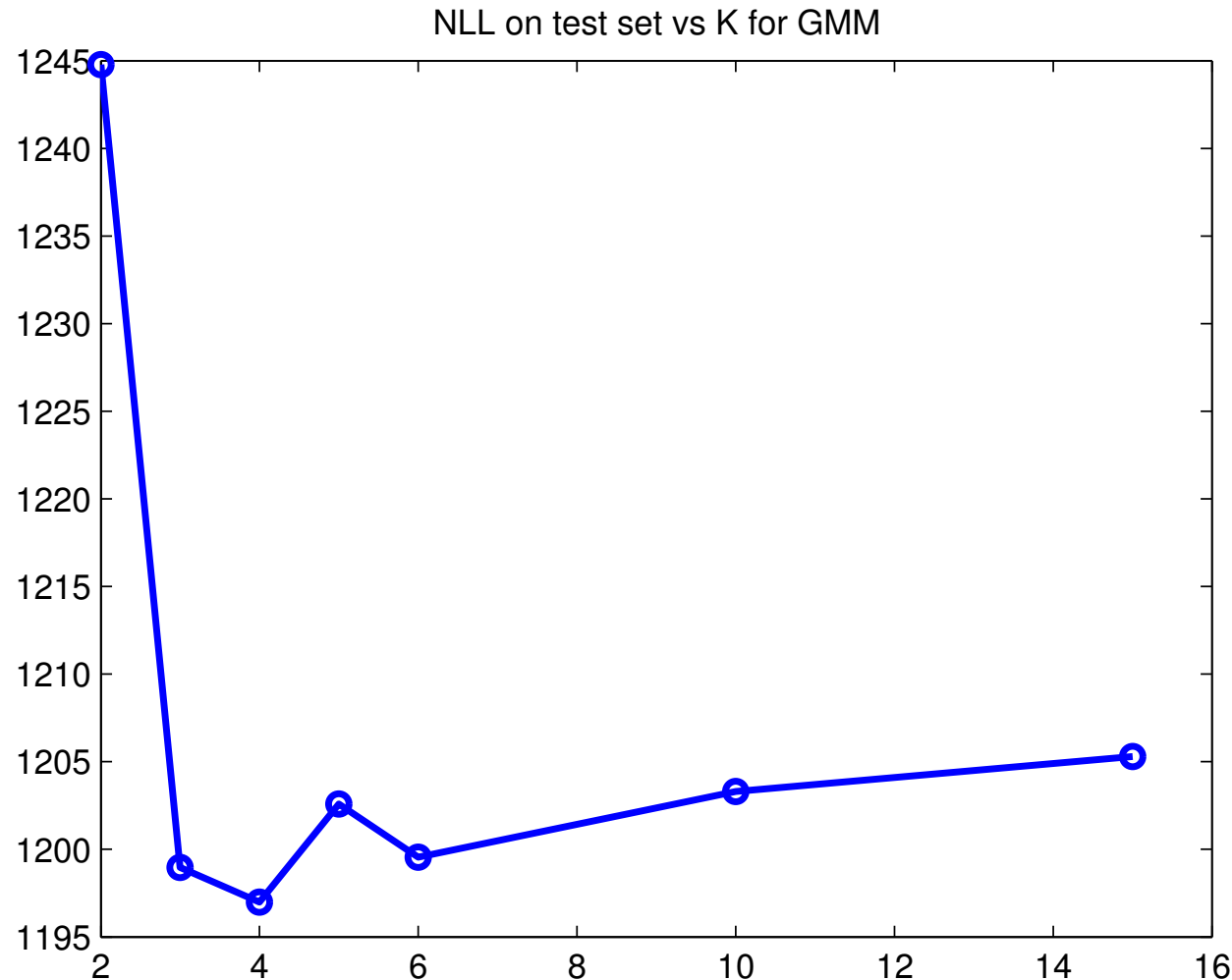
Model Selection for Non-Probabilistic Models

- Supervised learning setting is easy
 - Cross-validation works
- Unsupervised learning is a different story
 - K-means reconstruction error decreases on test set with model complexity!



- Adding more centroids → tiles the space more densely
 - Test vectors more likely to find a good close prototype for larger K
 - Issue not usually highlighted in machine-learning literature

Model Selection for Probabilistic Models



Standard cross validation using the negative test log likelihood (NLL) does give us the right shape

Conclusions

- Latent variable models
 - Learning is hard due to dependencies among parameters
 - Point estimation via MLE or MAP
- Mixture Models
 - » Clustering, density modelling
 - » GMMs, Mixture of products of Bernoulli distributions
- EM algorithm
 - » Optimisation of the expected complete data log-likelihood
 - » Iterative algorithm
 - » Susceptible to local minima
- Reading:
 - Murphy (MLaPP, 2012):
 - » Sec. 10.4
 - » Ch. 11 (except 11.4.5, 11.4.6, 11.4.8, 11.4.9, 11.6)